

KolleischSuen Lesson — 19 June 2026

Goal for today

Turn the generic Next.js starter page into the first recognizable KolleischSuen screen.

By the end of the 45-minute lesson, students should:

- understand the four users of the application;
- know the difference between the UI, server, and database;
- run the application locally;
- create at least one new page;
- see their changes immediately in the browser;
- understand why private keys must be protected.

Do not make real blockchain transfers or use production accounts and private keys during this lesson.

1. The problem and the four users — 8 minutes

Explain the current paper-ticket process and its digital replacement:

1. The cashier receives euros and issues digital event tokens.
2. The customer owns and spends the tokens.
3. The vendor creates a payment request and receives tokens.
4. The organizer checks whether all totals match.

Ask the students:

- What information does the cashier need?
- What should the customer check before paying?
- What does the vendor need to show?
- What totals does the organizer need?

Important examples:

- The cashier must clearly see the recipient, amount, and token.
- The customer must clearly see the vendor, amount, token, and memo before confirming.
- The vendor needs a simple payment request, eventually displayed as a QR code.
- The organizer needs issued and received totals.

2. How the web application is organized — 10 minutes

Introduce three parts of the application:

- **UI:** Pages that people see and use, such as `/cashier` and `/vendor`.
- **Server:** Validates requests and later communicates with Hive.
- **Database:** Stores booths, payment requests, and transfer records.

Explain that Next.js uses folders to create routes:

```
app/  
  page.tsx  
  cashier/  
    page.tsx  
  customer/  
    page.tsx  
  vendor/  
    page.tsx  
  organizer/  
    page.tsx
```

For example:

- `app/page.tsx` becomes `/`
- `app/vendor/page.tsx` becomes `/vendor`
- `app/cashier/page.tsx` becomes `/cashier`

Install the project dependencies:

```
npm install
```

Start the development server:

```
npm run dev
```

Keep that terminal running. Open a second terminal and launch the site:

```
Start-Process http://localhost:3000
```

Show the students that changing text in `app\page.tsx` causes the browser page to update.

3. Build the first visible feature — 17 minutes

Create folders for the four application roles:

```
New-Item -ItemType Directory -Force app\cashier  
New-Item -ItemType Directory -Force app\customer  
New-Item -ItemType Directory -Force app\vendor  
New-Item -ItemType Directory -Force app\organizer
```

Open the project in Visual Studio Code:

```
code .
```

Replace the default homepage with a simple role-selection screen containing links for:

- Cashier
- Customer
- Vendor
- Organizer

If there is enough time, create `app\vendor\page.tsx`:

```
export default function VendorPage() {  
  return (  
    <main className="p-8">  
      <h1 className="text-3xl font-bold">Vendor</h1>  
      <p>Create a payment request here.</p>  
    </main>  
  );  
}
```

Open the new page:

```
Start-Process http://localhost:3000/vendor
```

Explain what happened:

1. Next.js found the `app\vendor` folder.
2. It found the `page.tsx` file inside it.
3. It automatically created the `/vendor` URL.
4. React turned the TypeScript/JSX code into the page shown by the browser.

4. Data and security — 10 minutes

Open `prisma\schema.prisma`.

For today, explain only these three database models:

- `booth`: A place where food or drinks are sold.
- `payment_request`: The amount and recipient shown to a customer.
- `token_transfer`: A record of tokens being issued, paid, refunded, or corrected.

Validate the Prisma database description:

```
npx prisma validate
```

Format it consistently:

```
npx prisma format
```

The main security rule

A private key is like a password that can spend money. Never print it, commit it, screenshot it, or use a real one during class.

The README contains an older proposal where the browser signs transactions directly. The newer decision in [AGENTS.md](#) says that signing and broadcasting should happen through a local server API route. A submitted key may only be used temporarily in memory and must never be logged or stored by the server.

This demonstrates that software requirements can change as a team learns more about security.

Check the work

Show which files have changed:

```
git status --short
```

Show the actual code changes:

```
git diff
```

Check the code for common mistakes:

```
npm run lint
```

Expected result

At the end of the lesson:

- the Next.js development server is running;
- students can open the application in a browser;
- the four roles are understood;
- the four route folders exist;
- the homepage is becoming a role-selection screen;
- at least the [/vendor](#) page works;
- students understand that real accounts, tokens, and private keys must not be used during early development.

Visible progress is more important today than implementing blockchain payments. The next lesson can turn the vendor page into a form that creates and validates a versioned payment request.